The Journal of Open Source Software

torchvision-customizer: A Modular Framework for Building and Customizing CNN Architectures in PyTorch

Ahsan Umar [1¶]

1 Independent Researcher, Pakistan ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 21 December 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

`torchvision-customizer` is an open-source Python library that provides a flexible, modular framework for building, customizing, and fine-tuning convolutional neural network (CNN) architectures in PyTorch. The library addresses a fundamental challenge in deep learning research: the difficulty of modifying pre-trained model architectures without extensive code changes, copy-pasting model source code, or using fragile monkey-patching techniques.

The framework introduces a **three-tier API architecture** designed to accommodate different user needs:

1. **Component Registry** — A centralized catalog of 40+ building blocks for discovery and instantiation
2. **Architecture Recipes** — Declarative YAML configurations with macros, inheritance, and schema validation
3. **Model Composer** — A fluent Python API using the >> operator for intuitive model construction

Version 2.1 introduces the **Hybrid Builder**, which enables researchers to load any pre-trained torchvision model and apply targeted modifications—such as injecting attention mechanisms, replacing layers, or modifying block structures—while automatically preserving compatible pre-trained weights. Version 2.1.1 adds a **Training API** for quick experimentation on MNIST, CIFAR-10, and CIFAR-100 datasets.

Statement of Need

Deep learning research frequently requires architectural experimentation. Researchers may wish to investigate adding attention mechanisms to existing architectures (Hu et al., 2018; Wang et al., 2020; Woo et al., 2018), replacing normalization strategies, or combining components from different architectural families (He et al., 2016; Zhuang Liu et al., 2022; Tan & Le, 2019). However, the current ecosystem presents significant friction for such modifications.

Libraries like **torchvision** (PyTorch Contributors, 2016) provide well-tested implementations with pre-trained weights, but modifying these models—for instance, injecting Squeeze-and-Excitation blocks into a ResNet—requires understanding internal implementation details and manually modifying the forward pass. The **timm** library (Wightman, 2019) offers extensive model variants but focuses on providing ready-to-use architectures rather than enabling user-driven customization.

Current approaches to architectural modification include:

- **Code duplication**: Copying model source code into projects and modifying directly, which creates maintenance burden and makes it difficult to incorporate upstream improvements
- **Monkey-patching**: Runtime modification of model behavior, which is fragile, difficult to reproduce, and error-prone
- **Manual reimplementation**: Writing custom model classes from scratch, requiring substantial boilerplate for channel tracking and component interconnection

`torchvision-customizer` addresses these limitations through a principled approach to model composition and customization, enabling:

- **Block-level customization** of pre-trained models through a patch-based modification system
- **Automatic weight preservation** when modifying architectures, with graceful handling of shape mismatches
- **Declarative model definition** through YAML recipes for reproducible experiments
- **Rapid prototyping** of custom architectures with automatic channel dimension tracking
- **Built-in training utilities** for quick experimentation on standard datasets

Key Features

Hybrid Builder

The Hybrid Builder enables customization of pre-trained torchvision models without requiring users to understand internal implementation details. Users specify a backbone name, optional pre-trained weights, and a dictionary of patches describing modifications. The framework handles architecture decomposition, patch application, weight preservation, and head replacement automatically.

Supported backbone families include:

Family	Models	Reference
ResNet	resnet18, 34, 50, 101, 152	He et al. (2016)
Wide ResNet	wide_resnet50_2, 101_2	Zagoruyko & Komodakis (2016)
ResNeXt	resnext50_32x4d, 101_32x8d	Xie et al. (2017)
EfficientNet	B0-B7, V2 variants	Tan & Le (2019); Tan & Le (2021)
ConvNeXt	tiny, small, base, large	Zhuang Liu et al. (2022)
MobileNet	v2, v3_small, v3_large	Howard et al. (2017); Sandler et al. (2018)
VGG	11, 13, 16, 19 (with/without BN)	Simonyan & Zisserman (2014)
DenseNet	121, 169, 201, 161	Huang et al. (2017)
Vision Transformer	vit_b_16, vit_b_32, vit_l_16, vit_l_32	Dosovitskiy et al. (2021)
Swin Transformer	swin_t, swin_s, swin_b	Ze Liu et al. (2021)

Building Blocks

The library provides over 40 building blocks organized into functional categories:

- **Attention mechanisms**: Squeeze-and-Excitation (Hu et al., 2018), CBAM (Woo et al., 2018), and Efficient Channel Attention (Wang et al., 2020)
- **Convolution blocks**: Standard, depthwise separable (Howard et al., 2017), and mobile inverted bottleneck (MBConv) variants (Tan & Le, 2019)

DRAFT

- **Residual blocks**: Basic, bottleneck (He et al., 2016), wide (Zagoruyko & Komodakis, 2016), and grouped convolution (Xie et al., 2017) variants
- **Regularization**: Stochastic depth (DropPath) (Huang et al., 2016) for training regularization
- **Activations**: Including Mish (Misra, 2019) and standard PyTorch activations
- **Pooling**: Including Generalized Mean Pooling (GeM) (Radenović et al., 2019) with learnable parameters

Weight Utilities

The framework includes utilities for **partial weight loading** that gracefully handle architectural mismatches. When source and target architectures differ, the utilities:

1. Identify matching parameters by name
2. Verify shape compatibility
3. Load only the compatible subset
4. Initialize new parameters using specified schemes (Kaiming, Xavier, or zero initialization)
5. Produce detailed reports indicating loading success rates and any issues encountered

Training API

Version 2.1.1 introduces built-in training utilities for quick experimentation:

- **Trainer class** with methods for MNIST, CIFAR-10, CIFAR-100, and custom datasets
- **Optimizers**: Adam, AdamW, SGD with momentum
- **Schedulers**: Cosine annealing, step decay, OneCycle
- **Automatic device selection** (CPU/CUDA)
- **Training metrics** with history tracking and summaries

YAML Recipes

Models can be defined declaratively using YAML configuration files with:

- **Macro expansion** for reusable values
- **Recipe inheritance** from base configurations
- **Schema validation** for early error detection

Command-Line Interface

The CLI (`tvc` command) provides access to common operations:

- `tvc build` — Build model from YAML recipe
- `tvc benchmark` — Benchmark model performance
- `tvc validate` — Validate recipe configuration
- `tvc export` — Export to ONNX/TorchScript
- `tvc list-backbones` — List available backbone architectures
- `tvc list-blocks` — List available building blocks

Comparison with Related Work

Feature	torchvision	timm	torchvision-customizer
Pre-trained models			*
Block-level customization	—	—	
Attention injection	—	—	
Declarative configuration	—	—	

DRAFT

Feature	torchvision	timm	torchvision-customizer
Fluent composition API	—	—	
Partial weight loading	—		
Command-line interface	—	—	
Recipe inheritance	—	—	
Built-in training API	—	—	

*Through Hybrid Builder integration with torchvision

The library complements existing tools by providing a customization layer that operates on top of torchvision models. Rather than reimplementing architectures, we leverage existing torchvision implementations and provide mechanisms for targeted modification, ensuring compatibility with the broader PyTorch ecosystem.

Target Audience

`torchvision-customizer` is designed for:

- **Academic researchers** investigating attention mechanisms, normalization techniques, or novel architectural components who need to inject modifications into established architectures while leveraging pre-trained representations
- **Applied practitioners** developing domain-specific computer vision solutions who need to efficiently adapt pre-trained models with selective layer freezing for effective transfer learning
- **Educators** teaching CNN architecture design who can use the explicit composition syntax to make design choices visible and modifiable
- **AutoML researchers** requiring programmatic architecture generation who can leverage the registry and composer APIs for integration with architecture search algorithms

Software Availability

`torchvision-customizer` is open-source software released under the MIT license. The source code is hosted on GitHub at https://github.com/codewithdark-git/torchvision-customizer. The repository includes comprehensive documentation, API reference, user guides, and example scripts. Requirements include Python 3.11+, PyTorch 2.5+, and torchvision 0.20+.

Acknowledgments

We thank the PyTorch development team for creating and maintaining the foundational framework upon which this work builds. We also acknowledge the broader open-source community whose feedback has shaped the library's development.

References

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations (ICLR)*.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. https://doi.org/10.1109/CVPR.2016.90

DRAFT

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv Preprint arXiv:1704.04861*.

Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 7132–7141. https://doi.org/10.1109/CVPR.2018.00745

Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4700–4708. https://doi.org/10.1109/CVPR.2017.243

Huang, G., Sun, Y., Liu, Z., Sedra, D., & Weinberger, K. Q. (2016). Deep networks with stochastic depth. *Proceedings of the European Conference on Computer Vision (ECCV)*, 646–661. https://doi.org/10.1007/978-3-319-46493-0_39

Liu, Ze, Lin, Y., Cao, Y., Hu, Y., Han Wei, Zhang, Z., Lin, S., & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 10012–10022. https://doi.org/10.1109/ICCV48922.2021.00986

Liu, Zhuang, Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A ConvNet for the 2020s. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 11976–11986. https://doi.org/10.1109/CVPR52688.2022.01167

Misra, D. (2019). Mish: A self regularized non-monotonic activation function. *arXiv Preprint arXiv:1908.08681*.

PyTorch Contributors. (2016). *Torchvision*. https://github.com/pytorch/vision.

Radenović, F., Tolias, G., & Chum, O. (2019). Fine-tuning CNN image retrieval with no human annotation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *41*(7), 1655–1668. https://doi.org/10.1109/TPAMI.2018.2846566

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4510–4520. https://doi.org/10.1109/CVPR.2018.00474

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv Preprint arXiv:1409.1556*.

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *Proceedings of the 36th International Conference on Machine Learning (ICML)*, *97*, 6105–6114.

Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller models and faster training. *Proceedings of the 38th International Conference on Machine Learning (ICML)*, *139*, 10096–10106.

Wang, Q., Wu, B., Zhu, P., Li, P., Zuo, W., & Hu, Q. (2020). ECA-Net: Efficient channel attention for deep convolutional neural networks. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 11534–11542. https://doi.org/10.1109/CVPR42600.2020.01155

Wightman, R. (2019). *PyTorch Image Models*. https://github.com/rwightman/pytorch-image-models. https://doi.org/10.5281/zenodo.4414861

Woo, S., Park, J., Lee, J.-Y., & Kweon, I. S. (2018). CBAM: Convolutional block attention module. *Proceedings of the European Conference on Computer Vision (ECCV)*, 3–19. https://doi.org/10.1007/978-3-030-01234-2_1

Xie, S., Girshick, R., Dollár, P., Tu, Z., & He, K. (2017). Aggregated residual transformations for deep networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern*

DRAFT

Recognition (CVPR), 1492–1500. https://doi.org/10.1109/CVPR.2017.634

Zagoruyko, S., & Komodakis, N. (2016). Wide residual networks. *arXiv Preprint arXiv:1605.07146*.

DRAFT